

Môn học: Các Công nghệ Lập trình Hiện đại

Mục đích: giúp nhóm chọn đúng công nghệ, biết phần kiến thức bắt buộc phải nắm, các nội dung có thể học thêm và cách chuẩn bị minh chứng cho đề án.

Phiên bản dành cho sinh viên – 09/2026

Mô hình ba tầng — áp dụng cho mọi track

Tầng	Nội dung	Ví dụ	Yêu cầu đối với nhóm
Tầng 1 Bản chất công nghệ	Những cơ chế riêng làm công nghệ đó khác với công nghệ khác. Quy định riêng cho từng công nghệ tại Phần F.	Next.js: Server vs Client Component, caching/revalidation. NestJS: Module, Provider, Guard, Pipe.	Phần bắt buộc lõi là điều kiện cần, thiếu là chưa đạt phần công nghệ. Phần chọn thêm do nhóm tự chọn theo sức.
Tầng 2 Kỹ nghệ phần mềm	Chuẩn nghề nghiệp chung, áp dụng theo đặc thù nền tảng. Quy định tại Phần D.	Quản lý phiên bản, kiểm thử, bảo mật, tái lập môi trường, tài liệu.	Bắt buộc, nhưng nội dung cụ thể thay đổi theo nền tảng (code hay low-code).
Tầng 3 Nâng cao và mở rộng	Kỹ thuật tự chọn để tạo chiều sâu và khác biệt. Quy định tại Phần C.	Kéo thả, canvas, 3D, realtime, hàng đợi, cache, quan sát hệ thống.	Tự chọn theo sức, mục đích và nhu cầu thật của bài toán. Không có ngưỡng số lượng chung; số lượng chỉ có ý nghĩa khi đi cùng độ sâu, tính phù hợp và chất lượng hiện thực hoá.

HỒ SƠ CỦA MỖI NHÓM

1 công nghệ chính (Phần A) + phần BẮT BUỘC LỖI của công nghệ đó (Phần F) + các mục CHỌN THÊM của công nghệ đó, tùy sức và tùy mục đích bài toán + kỹ thuật nâng cao tự chọn (Phần C) + chuẩn kỹ nghệ phần mềm theo đặc thù nền tảng (Phần D).

Nhóm làm đề tài AI có thêm một hồ sơ đề tài (Phần B), vì với AI thì hình dạng bài toán quyết định quy trình bắt buộc nhiều hơn là tên thư viện.

CÁCH CHỌN PHẠM VI HỌC TẬP

Tài liệu nêu phần BẮT BUỘC LỖI của từng công nghệ và các hướng CHỌN THÊM. Nhóm tự chọn công nghệ, bài toán và phần mở rộng phù hợp với năng lực, thời gian và mục tiêu. Không cần làm thật nhiều; cần làm đúng, hiểu rõ và chứng minh được những gì đã sử dụng. Điểm cuối cùng do chất lượng đồ án quyết định sau khi nhóm qua công thực hành tại chỗ.

Cách đọc danh mục

Ký hiệu	Ý nghĩa
☆☆☆	Dễ vào. Nhóm nền yếu vẫn hoàn thành được, nhưng phải nâng yêu cầu ở Tầng 3 để đủ chiều sâu.
★★☆	Vừa sức. Lựa chọn an toàn cho phần lớn nhóm 3–4 sinh viên trong 12 tuần.
★★★	Khó. Phù hợp nhóm có nền vững và khả năng tự học tốt.
Cột "Loại"	Phân loại học thuật của đối tượng: framework, meta-framework, thư viện, nền tảng, giao thức, kiến trúc, công cụ... Việc gọi đúng tên loại là một phần của yêu cầu học thuật đối với báo cáo.
Mức thuận lợi khi thực hành tại chỗ	Mức độ thuận lợi khi ra bài thực hành 45–60 phút: Dễ (chạy ngay trên trình duyệt hoặc terminal), Vừa (cần môi trường chuẩn bị trước), Khó (phụ thuộc thiết bị, dịch vụ ngoài, hoặc thời gian biên dịch lâu).

ĐỊNH NGHĨA "CÔNG NGHỆ MỚI" TRONG MÔN NÀY

Mới = mới với sinh viên, tức là chưa được dạy chính khoa ở bất kỳ môn nào trước đó — không nhất thiết là công nghệ vừa ra đời.

Ràng buộc kèm theo: mỗi công nghệ không quá 4 nhóm; các nhóm cùng công nghệ phải khác miền bài toán.

I. BỐI CẢNH 2026

I.1. Thị trường lao động IT Việt Nam

- Ba vị trí có nhu cầu tuyển cao nhất: lập trình viên web (khoảng 45%), kỹ sư backend (37,4%), kỹ sư AI (30%).
- Nhóm khó tuyển nhất: kỹ sư AI (12,2%), bridge engineer (9,92%), solution architect (9,16%), lập trình viên mobile (7,63%).
- 55% doanh nghiệp cho biết khó khăn lớn là thiếu ứng viên có kỹ năng cao. Vì vậy môn học nhấn mạnh khả năng hiểu, giải thích và vận dụng công nghệ, không chỉ khả năng tạo ra một sản phẩm chạy được.
- Khoảng 40,7% doanh nghiệp đặt AI vào chiến lược phát triển; riêng khối IT–phần mềm là 58%.

I.2. Xu hướng kỹ thuật đáng chú ý

Xu hướng	Ý nghĩa với môn học
Meta-framework là điểm khởi đầu mặc định	Next.js và Nuxt dần thay thế việc dựng dự án từ thư viện thuần. Server Actions và server functions đưa một phần backend vào ngay repo frontend.
TypeScript	Các đồ án thuộc hệ sinh thái JavaScript ở cả frontend và backend được khuyến nghị hoặc yêu cầu dùng TypeScript , trừ khi mục tiêu đề tài yêu cầu JavaScript thuần (ví dụ nhóm muốn nghiên cứu chính bản thân ngôn ngữ).
React Compiler (bản 1.0, 10/2025)	Tự động lo việc ghi nhớ (memo hoá) khi biên dịch. Next.js 16 hỗ trợ ổn định nhưng không bật mặc định — phải cài babel-plugin-react-compiler và đặt reactCompiler: true trong next.config. Là chủ đề đo đạc tốt, xem mục IX.

Xu hướng	Ý nghĩa với môn học
Hệ TanStack	Query, Router, Table, Form đang trở thành lớp logic phổ biến của frontend. Đây là thư viện/hệ sinh thái , nên tính là thành phần của stack chứ không phải một công nghệ chính để đăng ký riêng.
MCP (Model Context Protocol)	Một giao thức mở chuẩn hoá cách ứng dụng AI kết nối với hệ thống bên ngoài (khoảng 97 triệu lượt tải SDK mỗi tháng). MCP không phải một AI framework; cần gọi đúng đây là một giao thức/chuẩn mở.
LangGraph 1.0 (10/2025)	Framework/runtime điều phối tác tử dạng đồ thị, xây trên ba khái niệm State – Nodes – Edges. Phù hợp nhóm khá.
PostgreSQL + pgvector	Lựa chọn mặc định được khuyến nghị trong môn học cho đồ án quy mô vừa và nhỏ: pgvector là extension của PostgreSQL, hỗ trợ tìm láng giềng gần chính xác và xấp xỉ (HNSW, IVFFlat), cho phép nhóm tìm hiểu cả vector search lẫn dữ liệu quan hệ trong cùng một hệ. Đây là lựa chọn được khuyến nghị trong phạm vi môn học, không phải nhận định tuyệt đối về thị trường.
Khoảng trống đánh giá (evals)	89% đội có tác tử chạy thật đã làm observability nhưng chỉ 52% có bộ đánh giá. Sinh viên làm được phần đánh giá bằng số liệu sẽ rất nổi bật.
Nuxt 3 hết vòng đời 31/7/2026	Tài liệu chính thức đã hướng sang Nuxt 4. Nhóm chọn hệ Vue phải chọn Nuxt 4, không chọn Nuxt 3.

II. PHÂN LOẠI SÁU NHÓM

Danh mục này không cố ép mọi thứ thành từ "công nghệ". Sáu nhóm dưới đây khác nhau về bản chất và có vai trò khác nhau trong hồ sơ của nhóm.

Phần	Tên nhóm	Bản chất	Vai trò trong hồ sơ nhóm
A	Công nghệ / framework / nền tảng chính	Framework, meta-framework, SDK, nền tảng low-code, runtime	Chọn làm công nghệ chính. Mỗi nhóm chọn đúng một công nghệ chính phù hợp với bài toán.
B	Kiến trúc và chủ đề chuyên sâu	Pattern, kiến trúc, cách triển khai — không phải phần mềm cụ thể	Hồ sơ đề tài. Bắt buộc với track AI, tùy chọn với track khác.
C	Kỹ thuật nâng cao tự chọn	Nhóm kỹ thuật chuyên môn	Tầng 3. Tự chọn theo bài toán để tạo chiều sâu; không có số lượng tối thiểu chung. Độ sâu và tính phù hợp được phản ánh trong điểm chất lượng đồ án.
D	Chuẩn kỹ nghệ phần mềm	Thực hành nghề nghiệp (practice)	Tầng 2. Bắt buộc, nội dung điều chỉnh theo đặc thù nền tảng.
E	Hạ tầng và công nghệ hỗ trợ	DBMS, cache, CI, công cụ kiểm thử, chuẩn quan sát	Bổ trợ. Không chọn làm công nghệ chính.

Phần	Tên nhóm	Bản chất	Vai trò trong hồ sơ nhóm
F	Yêu cầu bản chất theo từng công nghệ	Đặc tả yêu cầu	Tầng 1. Điều kiện cần để được công nhận phần công nghệ.

Ví dụ minh họa việc gọi đúng loại

Tên	Loại	Ghi chú
Next.js 16	Meta-framework	Có thể chọn làm công nghệ chính
React 19	Thư viện giao diện (UI library)	Có thể chọn làm công nghệ chính, thường đi kèm một build tool và một bộ thư viện
Vite	Công cụ build	Không đứng riêng làm công nghệ chính
TanStack Query / Router	Thư viện / hệ sinh thái	Thành phần của stack
NestJS	Backend framework	Có thể chọn
Flutter	Cross-platform UI framework / SDK	Có thể chọn
LangGraph	Agent orchestration framework / runtime	Có thể chọn
MCP	Giao thức / chuẩn mở	Không cùng loại với framework. Chọn dưới dạng hồ sơ "MCP Server" ở Phần B, dùng MCP SDK.
RAG	Kiến trúc / pattern	Không phải một phần mềm. Là hồ sơ đề tài ở Phần B.
n8n	Nền tảng tự động hoá quy trình	Có thể chọn
AppSheet	Nền tảng no-code	Có thể chọn, nhưng có yêu cầu bản chất riêng — xem F5
Supabase / Firebase	Nền tảng backend (BaaS/PaaS)	Nên dùng làm công nghệ hỗ trợ, không làm đề tài chính
Docker	Công nghệ container	Hạ tầng hỗ trợ (Phần E)
OWASP Top 10	Khung kiến thức bảo mật	Thuộc chuẩn kỹ nghệ phần mềm (Phần D)

III. PHẦN A — DANH MỤC CÔNG NGHỆ, THƯ VIỆN VÀ NỀN TẢNG ĐỂ LỰA CHỌN

Đây là danh sách những gì một nhóm được chọn làm công nghệ chính. Yêu cầu bản chất tương ứng của từng mục nằm ở Phần F.

A1. Web frontend và meta-framework

Nhu cầu tuyển dụng cao và thuận lợi cho bài thực hành tại chỗ vì phần lớn nội dung có thể chạy trực tiếp trên trình duyệt.

Tên	Loại	Độ khó	Nền cần có	Ghi chú 2026 · mức thuận lợi khi thực hành tại chỗ
Next.js 16	Meta-framework	★★☆	JS/TS, React cơ bản	Phổ biến nhất hiện nay. App Router dựa trên Server Components và server functions. React Compiler hỗ trợ ổn định nhưng phải bật thủ công. · Thực hành tại chỗ: Dễ
React 19 + Vite + TanStack	Thư viện UI + công cụ build + hệ thư viện	★★☆	JS/TS	Nhẹ hơn meta-framework, buộc sinh viên hiểu React sâu hơn. Khi chọn phải nêu rõ stack gồm những gì. · Thực hành tại chỗ: Dễ
Nuxt 4 (Vue 3)	Meta-framework	★★☆	HTML/CSS/JS	Cú pháp dễ vào nhất trong ba hệ lớn. Đăng ký Nuxt 4, không chọn Nuxt 3 vì Nuxt 3 hết vòng đời hỗ trợ ngày 31/7/2026. · Thực hành tại chỗ: Dễ
SvelteKit	Meta-framework	★★☆	JS/TS	Ít việc ở VN hơn nhưng dạy khái niệm reactivity rất rõ và code ngắn. Hợp nhóm khá muốn đề tài khác biệt. · Thực hành tại chỗ: Dễ
Astro	Meta-framework hướng nội dung	★☆☆	HTML/CSS/JS	Kiến trúc islands, tối ưu web nội dung. Dễ hoàn thành nhưng phải bù chiều sâu bằng Tầng 3 hoặc ghép backend. · Thực hành tại chỗ: Vừa
Angular	Framework	★★★	TypeScript, OOP	Vẫn nhiều trong doanh nghiệp và outsourcing. Đốc học cao và nặng khái niệm. · Thực hành tại chỗ: Vừa

A2. Backend và API

Tên	Loại	Độ khó	Nền cần có	Ghi chú 2026 · mức thuận lợi khi thực hành tại chỗ
NestJS	Backend framework	★★☆	TypeScript, OOP	Cấu trúc theo module rất thuận cho việc học và trình bày kiến trúc. Ghép Prisma hoặc Drizzle dễ hơn TypeORM. Lựa chọn an toàn nhất nhóm này. · Thực hành tại chỗ: Dễ
ASP.NET Core	Backend framework	★★☆	C#, SQL	Nhiều việc ở VN, đặc biệt khối ngân hàng và doanh nghiệp. Minimal API rút ngắn thời gian khởi động dự án. · Thực hành tại chỗ: Dễ
FastAPI	Backend / API framework	★★☆	Python	Ra kết quả nhanh; Pydantic giúp làm rõ validation và kiểu dữ liệu chặt. Lựa chọn tự nhiên nếu nhóm còn muốn ghép phần AI. · Thực hành tại chỗ: Dễ
Laravel	Backend framework	★★☆	PHP, SQL	Rất nhiều việc ở mảng doanh nghiệp vừa và nhỏ, agency tại VN. Hệ sinh thái đầy đủ sẵn. · Thực hành tại chỗ: Dễ
Spring Boot	Backend framework	★★★	Java, OOP	Nhiều việc ở ngân hàng, tài chính, doanh nghiệp lớn. Nặng cấu hình, khởi động chậm — cần tính vào thời lượng bài tại chỗ. · Thực hành tại chỗ: Vừa
Go + Gin / Echo / Fiber	Ngôn ngữ + web framework	★★★	Tư duy logic tốt	Xu hướng cho dịch vụ chịu tải. Ít khái niệm nhưng cú pháp lạ. Chỉ nên đăng ký nếu bài toán thật sự cần xử lý đồng thời. · Thực hành tại chỗ: Vừa
Express	Web framework tối giản (Node)	★★☆	JavaScript	Rất nhẹ. Nếu đăng ký thì bắt buộc bù bằng Tầng 3 (xác thực, cache, hàng đợi, kiểm thử) mới đủ khối lượng. · Thực hành tại chỗ: Dễ
Hono	Web framework chuẩn Web / đa runtime	★★☆	TypeScript	Tách riêng khỏi Express. Giá trị học nằm ở Web Standards Request/Response, typed bindings và triển khai đa runtime hoặc edge — gộp chung với Express là mất đúng phần đáng học này. · Thực hành tại chỗ: Dễ
Rust + Axum	Ngôn ngữ + web framework	★★★	Nền vững, tự học tốt	Ít việc ở VN nhưng giúp học sâu về ownership và type safety. Thời gian biên dịch lâu nên thực hành tại chỗ khó hơn. · Thực hành tại chỗ: Khó

A3. Ứng dụng đa nền tảng (Cross-platform Application)

Nhóm này bao gồm các công nghệ xây ứng dụng đa nền tảng cho mobile và/hoặc desktop. Với Kotlin Multiplatform và Tauri, cần làm rõ phần mã dùng chung và phần đặc thù nền tảng.

Tên	Loại	Độ khó	Nền cần có	Ghi chú 2026 · mức thuận lợi khi thực hành tại chỗ
Flutter	Cross-platform UI framework / SDK	★★☆	OOP	Lựa chọn cross-platform phổ biến nhất ở VN. Dart dễ học, tài liệu tiếng Việt nhiều. · Thực hành tại chỗ: Vừa
React Native + Expo	Cross-platform framework + nền tảng công cụ	★★☆	JavaScript, React	Tận dụng lại kiến thức React. Expo giúp không phải cài Xcode/Android Studio nặng — rất có lợi khi thi tại chỗ. · Thực hành tại chỗ: Vừa
Android + Kotlin + Jetpack Compose	Nền tảng native + toolkit UI khai báo	★★☆	Kotlin hoặc Java	Nếu nhóm chỉ nhắm Android thì học sâu hơn và sát tuyển dụng hơn cross-platform. · Thực hành tại chỗ: Vừa
.NET MAUI	Cross-platform framework	★★☆	C#	Hợp lý nếu nhóm đã học C#. Cộng đồng nhỏ hơn Flutter nên tự tìm tài liệu khó hơn. · Thực hành tại chỗ: Vừa
Kotlin Multiplatform + Compose Multiplatform	Công nghệ chia sẻ mã đa nền tảng	★★★	Kotlin, OOP	Đang lên nhanh: chia sẻ logic nhưng giữ giao diện gần native. Tài liệu tiếng Việt còn ít. · Thực hành tại chỗ: Khó
Tauri 2	Application framework (desktop + mobile)	★★★	Web + chút Rust	Ứng dụng nhẹ, đang thay dần Electron. Mô hình capability/permission là điểm học đáng giá. · Thực hành tại chỗ: Khó

A4. Thư viện, runtime và SDK cho ứng dụng AI

Nhóm AI chọn một mục ở đây làm công nghệ chính, và một hồ sơ đề tài ở Phần B để xác định quy trình bắt buộc.

Bản đồ tầng của một ứng dụng AI

Nhóm không cần làm hết các tầng, nhưng cần biết mình đang làm ở tầng nào và bỏ qua tầng nào.

Tầng	Vai trò	Công nghệ	Nội dung học được
1	Mô hình và cách gọi	SDK của OpenAI / Gemini / Claude; Ollama, llama.cpp; Hugging Face Transformers	Tham số, streaming, đếm token, tính chi phí
2	Kết nối công cụ	Function calling; MCP SDK	Cho mô hình gọi hệ thống của mình một cách có kiểm soát
3	Trí nhớ và tri thức	sentence-transformers; pgvector, Qdrant, Chroma; LlamaIndex	Nạp dữ liệu riêng, lập chỉ mục, truy hồi đúng đoạn

Tầng	Vai trò	Công nghệ	Nội dung học được
4A	Kiểm soát đầu ra có cấu trúc	Structured output của SDK; Instructor; Pydantic AI	Ép mô hình trả về đúng lược đồ. Đây là trích xuất và ràng buộc kiểu , không phải điều phối — hai việc khác nhau, thường bị gộp nhầm.
4B	Điều phối luồng và tác tử	LangChain; LangGraph	Ghép nhiều bước thành một luồng có trạng thái và rẽ nhánh
5	Đánh giá và quan sát	Langfuse; OpenTelemetry; bộ eval tự viết	Đo chất lượng bằng số; tìm lại được ca sai để phân tích
6	An toàn	Kiểm tra đầu vào và đầu ra; giới hạn quyền công cụ; môi trường cách ly	Chống prompt injection, không để mô hình làm quá quyền

Cách dùng bản đồ AI: mọi nhóm cần biết tầng mô hình/cách gọi và tầng đánh giá. Các tầng còn lại chỉ áp dụng khi bài toán cần: dữ liệu riêng → tầng tri thức; gọi công cụ/agent → tầng kết nối và điều phối; an toàn → áp dụng theo mức rủi ro của ứng dụng.

Tên	Loại	Độ khó	Nền tảng cơ	Ghi chú 2026 · mức thuận lợi khi thực hành tại chỗ
SDK mô hình (OpenAI / Gemini / Claude)	SDK / client library	★☆☆	Python hoặc TS	Điểm khởi đầu của mọi nhóm AI. Tham số, streaming, đếm token, tính chi phí, xử lý lỗi và giới hạn tần suất. · Thực hành tại chỗ: Dễ
LangGraph	Agent orchestration framework / runtime	★★☆	Python khá	Bản 1.0 từ 10/2025. Ba khái niệm nền: State, Nodes, Edges. Cảnh báo: dễ lạm dụng — nhiều bài toán chỉ cần gọi SDK trực tiếp. · Thực hành tại chỗ: Vừa
LlamaIndex	Thư viện lập chi mục và truy hồi dữ liệu	★★☆	Python	Chuyên cho bài toán dữ liệu riêng: connector nhiều nguồn, cắt đoạn, retriever, query engine. Ít việc phải tự viết hơn LangChain. · Thực hành tại chỗ: Vừa
MCP SDK (Python / TypeScript)	SDK triển khai một giao thức mở	★★☆	API, JSON	Bản thân MCP là giao thức, không phải framework. SDK là thứ sinh viên dùng để hiện thực hoá một server. · Thực hành tại chỗ: Dễ
Ollama / llama.cpp	Runtime chạy mô hình cục bộ	★★☆	Docker, dòng lệnh	Chạy mô hình mở ngay trên máy. Chỉ cần biết chọn bản lượng tử hoá phù hợp RAM, không cần hiểu toán đằng sau. · Thực hành tại chỗ: Vừa

Tên	Loại	Độ khó	Nền cần có	Ghi chú 2026 · mức thuận lợi khi thực hành tại chỗ
Hugging Face Transformers	Thư viện mô hình	★★★	Python, ML cơ bản	Tải và chạy mô hình mở qua pipeline; hiểu tokenizer và giới hạn ngữ cảnh. Không yêu cầu huấn luyện từ đầu. · Thực hành tại chỗ: Vừa
scikit-learn	Thư viện học máy	★★☆	Python, dữ liệu bảng	Vẫn là lựa chọn đúng cho dự đoán và phân loại trên dữ liệu bảng. Dùng hàm đánh giá có sẵn, không tự cài đặt thuật toán. · Thực hành tại chỗ: Dễ
Ultralytics YOLO	Thư viện mô hình thị giác máy tính	★★☆	Python	Nhận diện đối tượng. Suy luận và huấn luyện lại nhẹ bằng API có sẵn. Trực quan, dễ demo. · Thực hành tại chỗ: Vừa
faster-whisper	Thư viện mô hình xử lý tiếng nói	★★☆	Python	Chuyển giọng nói thành văn bản. Mô hình có sẵn đã tốt hơn nhiều so với tự huấn luyện trong 12 tuần. · Thực hành tại chỗ: Vừa
Langfuse	Nền tảng đánh giá và quan sát cho LLM	★★★	Python, tư duy đo đạc	Lưu vết từng lần gọi, gom thành bộ dữ liệu, chấm tự động, phát hiện hồi quy khi đổi prompt hoặc mô hình. · Thực hành tại chỗ: Vừa
Gradio / Streamlit	Thư viện dựng giao diện demo	★★☆	Python	Không chọn làm công nghệ chính. Dùng để nhóm AI có sản phẩm demo được mà không phải học thêm frontend. · Thực hành tại chỗ: Dễ

A5. Nền tảng tự động hoá và low-code / no-code

Tên	Loại	Độ khó	Nền cần có	Ghi chú 2026 · mức thuận lợi khi thực hành tại chỗ
n8n	Nền tảng tự động hoá quy trình (mã nguồn mở, tự triển khai được)	★★☆	API, JSON, logic rẽ nhánh	Mạnh lên nhờ các node AI. Là nền tảng duy nhất trong nhóm này tự triển khai được bằng Docker. · Thực hành tại chỗ: Vừa
Power Apps + Power Automate	Nền tảng low-code	★★☆	Tư duy CSDL, hàm bảng tính	Nhiều doanh nghiệp VN đã dùng Microsoft 365 nên kỹ năng này bán được. Cần tài khoản phù hợp. · Thực hành tại chỗ: Vừa

Tên	Loại	Độ khó	Nền cần có	Ghi chú 2026 · mức thuận lợi khi thực hành tại chỗ
Google AppSheet	Nền tảng no-code	★☆☆	Bảng tính, CSDL quan hệ	Bản chất là no-code — yêu cầu bản chất phải xoay quanh mô hình dữ liệu, biểu thức, bảo mật và tự động hoá của chính nền tảng, không phải quanh việc viết code. Xem F5. · Thực hành tại chỗ: Vừa
Supabase / Firebase	Nền tảng backend (BaaS / PaaS)	★☆☆	JS cơ bản, SQL với Supabase	Supabase gồm PostgreSQL, Auth, RLS, Storage, Realtime và Edge Functions. Nên dùng làm công nghệ hỗ trợ cho nhóm frontend, không làm đề tài chính. · Thực hành tại chỗ: Dễ

IV. PHẦN B — KIẾN TRÚC VÀ CHỦ ĐỀ CHUYÊN SÂU (HỒ SƠ ĐỀ TÀI)

Những mục dưới đây là kiến trúc, pattern hoặc cách triển khai — không phải phần mềm cụ thể, nên không đăng ký thay cho công nghệ chính. Với track AI, hồ sơ đề tài mới là thứ quyết định quy trình bắt buộc.

LƯU Ý KHI CHỌN HỒ SƠ AI

Mỗi dạng bài toán AI có quy trình kỹ thuật khác nhau. Nhóm làm MCP Server không phải thực hiện chunking/embedding nếu bài toán không cần; nhóm RAG không bắt buộc dùng agent. Hãy chọn đúng hồ sơ ở Phần B và thực hiện đúng quy trình của hồ sơ đó.

B1. Hồ sơ đề tài AI — quy trình bắt buộc của từng loại

Hồ sơ	Loại	Quy trình bắt buộc (Tầng 1 của hồ sơ)	Không bắt buộc
RAG (Retrieval-Augmented Generation)	Kiến trúc AI	Nạp dữ liệu → cắt đoạn → embedding và lập chỉ mục → truy hồi → gán nguồn (grounding, có trích dẫn) → sinh câu trả lời → đánh giá có thực nghiệm ở khâu truy hồi.	MCP. Tác tử. Chạy cục bộ.
MCP Server	Hiện thực một giao thức mở	Dựng server → định nghĩa tools và resources → lược đồ đầu vào (input schema) → tích hợp với một client thật → chọn transport → xử lý lỗi và giới hạn quyền → kiểm thử.	Vector DB nếu bài toán không cần. Chunking, embedding, rerank.
Model Serving	Kiến trúc triển khai	Nạp mô hình → tiền xử lý → suy luận → hậu xử lý → lược đồ API → xử lý đồng thời → đo độ trễ và thông lượng có số liệu.	RAG. Tác tử. Huấn luyện lại.

Hồ sơ	Loại	Quy trình bắt buộc (Tầng 1 của hồ sơ)	Không bắt buộc
Local LLM / Ollama	Cách triển khai	Chọn mô hình → chọn bản lượng từ hoá và runtime → đo mức dùng phần cứng → xử lý cửa sổ ngữ cảnh → đo độ trễ → đánh giá chất lượng → so sánh với API thương mại trên cùng bộ câu hỏi .	RAG — đừng bắt nhét vào nếu mục tiêu là nghiên cứu suy luận cục bộ.
Agentic system (LangGraph)	Kiến trúc ứng dụng	State → Nodes → Edges → định tuyến có điều kiện → lưu trạng thái và checkpoint → gọi công cụ. Có thể thêm human-in-the-loop.	Vector DB nếu tác tử không cần tra tài liệu.
Evaluation & Observability for LLM Applications	Thực hành kỹ nghệ	Bộ dữ liệu đánh giá → chọn thước đo → đường cơ sở (baseline) → chạy thực nghiệm → lưu vết (tracing) → phân loại kiểu lỗi → đánh giá hồi quy khi thay đổi.	Không nhất thiết phải sinh ra một ứng dụng. Sản phẩm là bộ đo.
Computer Vision (YOLO)	Kiến trúc ứng dụng thị giác	Bộ dữ liệu → tiền xử lý → suy luận hoặc huấn luyện lại → ngưỡng tin cậy và NMS → precision / recall / mAP → độ trễ và FPS → đưa thành dịch vụ.	RAG. Tác tử.

Ngưỡng tối thiểu cho hồ sơ Computer Vision: chỉ gọi một mô hình YOLO đã huấn luyện sẵn rồi vẽ khung bao quanh đối tượng là chưa đủ. Phải có bộ dữ liệu của nhóm, có chỉ số đánh giá, và có phần đưa mô hình thành dịch vụ gọi được.

B2. Chủ đề chuyên sâu cho các track khác (tuỳ chọn)

Chủ đề	Loại	Quy trình bắt buộc nếu chọn
Realtime collaboration	Kiến trúc ứng dụng	Kênh truyền → trạng thái chia sẻ → giải quyết xung đột (CRDT hoặc OT) → hiển thị người đang xem → khôi phục sau mất kết nối → kiểm chứng bằng hai máy chạy song song.
Offline-first	Kiến trúc ứng dụng	Lưu trữ cục bộ → hàng đợi thao tác → chiến lược đồng bộ → phát hiện và giải quyết xung đột → kiểm chứng bằng kịch bản ngắt mạng.
Edge deployment	Cách triển khai	Chọn runtime → giới hạn của môi trường edge → triển khai → đo độ trễ so với máy chủ truyền thống ở nhiều vị trí địa lý .
Event-driven / hàng đợi	Kiến trúc hệ thống	Sự kiện → producer/consumer → tính lũy đẳng (idempotency) → thử lại và hàng đợi thư chết → giám sát độ trễ hàng đợi.

V. PHẦN F — YÊU CẦU BẢN CHẤT THEO TỪNG CÔNG NGHỆ (TẦNG 1)

CÁCH ĐỌC YÊU CẦU CỦA TỪNG CÔNG NGHỆ

Mỗi công nghệ có hai cột yêu cầu. Cột "Bắt buộc lõi" là những cơ chế mà thiếu chúng thì coi như nhóm chưa thực sự dùng công nghệ đó — bắt buộc với mọi nhóm chọn công nghệ ấy. Cột "Chọn thêm" là phần nhóm tự chọn theo sức và theo mục đích của bài toán. Làm nhiều không tự động được điểm cao; giá trị nằm ở việc chọn đúng, làm đúng, làm sâu và chứng

minh được bằng code/kết quả chạy.

Câu hỏi tự kiểm ở cột cuối là câu nhóm phải tự trả lời được: "Nhóm em chọn công nghệ X — vậy cái gì trong đề án này là X, mà một dự án không dùng X sẽ không làm giống như vậy?"

F1. Web frontend và meta-framework

Công nghệ	Loại	BẮT BUỘC LỖI (thiếu là chưa dùng công nghệ này)	CHỌN THÊM tuỳ sức và mục đích của bài toán	Câu hỏi tự kiểm
Next.js 16	Meta-framework	App Router; ranh giới Server Component và Client Component; lấy dữ liệu phía máy chủ trong Server Component.	Caching và revalidation; Server Action hoặc Route Handler; route lồng nhau và động kèm layout / loading / error; streaming với Suspense; tối ưu ảnh và font của Next; middleware.	"Cái gì trong đề án này là Next.js mà một React SPA bình thường không làm giống như vậy?"
React 19 + Vite + TanStack	Thư viện UI + build tool + hệ thư viện	Component và hooks; phân biệt client state với server state; TanStack Query: cache và invalidation.	TanStack Router; refetch và cập nhật lạc quan; chia nhỏ gói và build bằng Vite; error boundary và Suspense; bật React Compiler và đo lại.	"Dữ liệu từ server và trạng thái giao diện — em quản lý khác nhau chỗ nào và vì sao?"
Nuxt 4 (Vue 3)	Meta-framework	Routing theo tệp; SSR và hydration; useFetch hoặc useAsyncData.	Composable tự viết; route middleware; server route của Nitro; route rules; chia sẻ trạng thái bằng useState của Nuxt.	"Trang này render ở đâu — máy chủ hay trình duyệt? Em chứng minh bằng cách nào?"
SvelteKit	Meta-framework	Cơ chế reactivity của Svelte; hàm load; ranh giới server và client.	Form actions; SSR hoặc prerender; hooks; stores; streaming.	"Ranh giới giữa code chạy trên server và trên client nằm ở đâu trong dự án của em?"
Astro	Meta-framework hướng nội dung	Kiến trúc islands; client directive và hydration một phần; chọn render tĩnh hay phía máy chủ.	Content collections; đo lường JavaScript gửi xuống trình duyệt; nhúng component của framework khác; view transitions.	"Trang này gửi xuống bao nhiêu KB JavaScript, và vì sao lại ít như vậy?"
Angular	Framework	Standalone component; DI và service; Router kèm guard.	Reactive Forms; signals; RxJS; HttpClient và interceptor; lazy load; change detection strategy.	"Vì sao chỗ này dùng service chứ không đặt logic thẳng trong component?"

F2. Backend và API

Công nghệ	Loại	BẮT BUỘC LỖI (thiếu là chưa dùng công nghệ này)	CHỌN THÊM tuỳ sức và mục đích của bài toán	Câu hỏi tự kiểm
NestJS	Backend framework	Module; Controller; Provider và Dependency Injection.	DTO và Pipe validation; Guard; Interceptor; Exception Filter; decorator tự viết; module động.	"Guard và Interceptor khác nhau ở đâu, và vì sao em đặt kiểm tra quyền ở Guard?"
ASP.NET Core	Backend framework	Middleware pipeline; DI có sẵn của framework; controller hoặc Minimal API.	EF Core và migration; authentication và authorization; model binding và validation; filter; hosted service; cấu hình theo môi trường.	"Thứ tự các middleware trong pipeline của em có ý nghĩa gì? Đảo hai cái thì sao?"
FastAPI	Backend / API framework	Pydantic model; hệ thống dependency; endpoint bất đồng bộ dùng đúng chỗ.	OpenAPI tự sinh; xử lý ngoại lệ tập trung; background task; middleware; response model; kiểm thử bằng TestClient.	"Endpoint nào của em nên là async và endpoint nào không? Vì sao?"

Công nghệ	Loại	BẮT BUỘC LỖI (thiếu là chưa dùng công nghệ này)	CHỌN THÊM tuỳ sức và mục đích của bài toán	Câu hỏi tự kiểm
Laravel	Backend framework	Route và controller; Eloquent và quan hệ; migration.	Form Request validation; middleware; policy hoặc gate; queue; event và listener; API resource.	"Vì sao dùng Form Request thay vì validate ngay trong controller?"
Spring Boot	Backend framework	IoC và Dependency Injection; REST controller và tầng service; Spring Data JPA.	Bean Validation; configuration và profile; Spring Security; AOP; exception handler toàn cục; test slice.	"Bean này được tạo ra lúc nào và ai tạo? Nếu em tự new nó thì mất gì?"
Go + Gin / Echo / Fiber	Ngôn ngữ + web framework	HTTP handler và middleware; context.Context; xử lý lỗi theo phong cách Go.	Goroutine và channel khi bài toán thực sự cần; interface và struct embedding; truy cập CSDL; tắt dịch vụ êm (graceful shutdown); table-driven test.	"context.Context trong handler của em dùng để làm gì? Bỏ đi thì hỏng chỗ nào?"
Express	Web framework tối giản (Node)	Middleware pipeline; router; xử lý lỗi bất đồng bộ đúng cách.	Validation; helmet, rate limit, CORS; truy cập CSDL; cấu trúc phân lớp; kiểm thử API.	"Một lỗi ném ra trong hàm async của em đi về đâu?"
Hono	Web framework chuẩn Web / đa runtime	Dùng Request và Response theo chuẩn Web; middleware; typed bindings.	Triển khai đa runtime hoặc lên edge; validator middleware; RPC client; streaming response.	"Cùng một mã nguồn chạy được trên Node và trên edge — nhờ cái gì?"
Rust + Axum	Ngôn ngữ + web framework	Ownership và borrowing ở những chỗ phù hợp; extractor; async với Tokio.	Chia sẻ trạng thái giữa handler; xử lý lỗi có kiểu; tower middleware; truy cập CSDL bất đồng bộ; kiểm thử.	"Trạng thái dùng chung giữa các request được chia sẻ an toàn nhờ cơ chế nào?"

F3. Ứng dụng đa nền tảng

Công nghệ	Loại	BẮT BUỘC LỖI (thiếu là chưa dùng công nghệ này)	CHỌN THÊM tuỳ sức và mục đích của bài toán	Câu hỏi tự kiểm
Flutter	Cross-platform UI framework / SDK	Widget tree và giao diện khai báo; quản lý trạng thái và cơ chế rebuild; điều hướng.	Future và Stream; plugin và platform channel; theming; kiểm thử widget; hoạt họa; lưu trữ cục bộ.	"Khi trạng thái đổi, những widget nào được dựng lại? Em kiểm chứng bằng gì?"
React Native + Expo	Cross-platform framework + nền tảng công cụ	Mô hình component native; hooks và state; điều hướng.	Các API thiết bị của Expo; lưu trữ cục bộ; hiệu năng danh sách dài; native module; build bằng EAS.	"Component này khi chạy trên máy thật biến thành cái gì?"
Android + Kotlin + Compose	Nền tảng native + toolkit UI khai báo	Composable và cơ chế recomposition; state hoisting; ViewModel kết hợp Flow.	Coroutine; vòng đời; Room hoặc DataStore; navigation; Material 3; WorkManager.	"Xoay màn hình thì dữ liệu đang nhập đi đâu? Vì sao?"
.NET MAUI	Cross-platform framework	XAML và C#; mô hình MVVM và data binding; điều hướng.	Command; DI; gọi API riêng của từng nền tảng; handler tuỳ biến; lưu trữ cục bộ.	"Binding hai chiều ở đây hoạt động thế nào?"
Kotlin Multiplatform	Công nghệ chia sẻ mã đa nền tảng	Phân biệt commonMain với source set riêng từng nền tảng; tầng logic dùng chung; cơ chế expect/actual.	Coroutine đa nền tảng; đo tỷ lệ mã dùng chung; Compose Multiplatform; dùng thư viện đa nền tảng.	"Bao nhiêu phần trăm mã dùng chung, và phần nào buộc phải viết riêng cho từng nền tảng?"
Tauri 2	Application framework	Frontend chạy trong WebView kết hợp lõi Rust; command và cơ chế IPC; mô hình capability và permission.	Đóng gói ứng dụng; cập nhật tự động; plugin; truy cập hệ thống tệp có kiểm soát.	"Frontend gọi được xuống hệ thống tệp nhờ cơ chế nào, và ai kiểm soát quyền đó?"

F4. Công nghệ AI — yêu cầu bản chất của công nghệ chính

AI KHÔNG ĐỒNG NGHĨA VỚI RAG

Nhóm AI chịu đồng thời hai bộ yêu cầu: bảng dưới đây (bản chất của công nghệ chính đã chọn) và quy trình bắt buộc của hồ sơ đề tài ở mục B1.

Các mục lỗi được chọn theo loại ứng dụng. Nhóm làm YOLO, scikit-learn hay MCP Server KHÔNG bị yêu cầu chunking, embedding hay vector store — đó là công việc của hồ sơ RAG. Điều bắt buộc với mọi hồ sơ chỉ có hai thứ: một luồng hoàn chỉnh từ đầu vào tới đầu ra, và một bảng số liệu đánh giá.

Công nghệ	Loại	BẮT BUỘC LỖI (thiếu là chưa dùng công nghệ này)	CHỌN THÊM tùy sức và mục đích của bài toán	Câu hỏi tự kiểm
SDK mô hình	SDK / client library	Gọi mô hình và đặt tham số có chủ đích, giải thích được ảnh hưởng; đếm token và tính chi phí mỗi lần gọi; xử lý lỗi và timeout.	Streaming; ràng buộc đầu ra theo lược đồ; function calling; đa phương thức; cache theo câu hỏi; giới hạn tần suất.	"Tăng temperature lên thì đầu ra của em đổi thế nào? Em đã thử chưa?"
LangGraph	Agent orchestration framework	Định nghĩa State; Nodes; Edges.	Định tuyến có điều kiện; lưu trạng thái và checkpoint; gọi công cụ; human-in-the-loop; giới hạn số vòng lặp.	"Về đồ thị của em ra. Nhánh nào chạy khi điều kiện này sai?"
LlamaIndex	Thư viện chỉ mục và truy hỏi	Nạp dữ liệu; lập chỉ mục; retriever và query engine.	Chiến lược cắt đoạn; gán nguồn cho câu trả lời; rerank; lọc theo metadata; nhiều nguồn dữ liệu.	"Đoạn văn bản nào đã được đưa vào ngữ cảnh để sinh ra câu trả lời này?"
MCP SDK	SDK của một giao thức mở	Định nghĩa tools và resources; lược đồ đầu vào; tích hợp với một client thật.	Chọn transport; prompts; xử lý lỗi và giới hạn quyền; kiểm thử; vòng đời server.	"MCP là gì — framework hay giao thức? Client biết server của em có tool nào bằng cách nào?"
Ollama / llama.cpp	Runtime cục bộ	Chọn mô hình và bản lượng từ hoá theo phần cứng; chạy suy luận cục bộ; đo độ trễ.	Cấu hình cửa sổ ngữ cảnh; đo mức dùng RAM và VRAM; so sánh chất lượng với mô hình qua API; dùng nhiều mô hình cho các tác vụ khác nhau.	"Vì sao chọn bản lượng từ hoá này mà không phải bản lớn hơn?"
Hugging Face Transformers	Thư viện mô hình	Tải mô hình và tokenizer; chạy suy luận qua pipeline.	Xử lý theo lô; chạy trên CPU hay GPU; giới hạn độ dài ngữ cảnh; tinh chỉnh nhẹ bằng API có sẵn của thư viện.	"Tokenizer làm gì với câu tiếng Việt này? Ra bao nhiêu token?"
scikit-learn	Thư viện học máy	Pipeline; tách tập huấn luyện và kiểm tra đúng cách; đánh giá bằng thước đo phù hợp bài toán.	Xử lý đặc trưng; kiểm định chéo; tinh chỉnh siêu tham số; phòng rò rỉ dữ liệu; lưu và nạp lại mô hình.	"Vì sao dùng thước đo này mà không dùng accuracy?"
Ultralytics YOLO	Thư viện mô hình thị giác	Bộ dữ liệu và gán nhãn; chạy suy luận; đo precision, recall, mAP.	Huấn luyện lại trên dữ liệu riêng; ngưỡng tin cậy và NMS; đo FPS; xuất ONNX; theo vết đối tượng.	"Hạ ngưỡng tin cậy xuống thì precision và recall đổi ra sao?"
Langfuse	Nền tảng đánh giá và quan sát	Ghi vết các lần gọi; xây bộ dữ liệu đánh giá; chạy thực nghiệm có đường cơ sở.	Phân loại kiểu lỗi; đánh giá hỏi quy khi đổi prompt hoặc mô hình; dùng mô hình chấm mô hình; theo dõi chi phí.	"Prompt mới tốt hơn prompt cũ — em chứng minh bằng số nào?"

F5. Nền tảng tự động hoá và low-code / no-code

LƯU Ý CHO LOW-CODE / NO-CODE

Không áp yêu cầu Docker hoặc viết code một cách máy móc cho Power Platform/AppSheet. Chiều sâu phải đến từ đúng bản chất nền tảng: mô hình dữ liệu, biểu thức/logic, phân quyền, tự động hoá, xử lý lỗi, tích hợp và kiểm thử.

E0 — Yêu cầu chung cho mọi nền tảng của track E

- Quy trình có trigger (sự kiện, lịch, hoặc thăm dò).
- Logic rẽ nhánh theo điều kiện.
- Biến đổi dữ liệu giữa các bước: chuẩn hoá định dạng, xử lý trường thiếu.
- Xử lý lỗi: phát hiện, ghi nhận, và có hành vi xác định khi một bước thất bại.
- Tích hợp từ hai hệ thống trở lên.

Công nghệ	Loại	BẮT BUỘC LỖI (thiếu là chưa dùng công nghệ này)	CHỌN THÊM tuỳ sức và mục đích của bài toán	Câu hỏi tự kiểm
E1. n8n	Nền tảng tự động hoá mã nguồn mở (tự triển khai được)	E0, cộng thêm: trigger và webhook; ánh xạ dữ liệu giữa các node; error workflow; tự triển khai bằng Docker .	Node code JavaScript hoặc Python; quản lý credential; subworkflow; cơ chế thử lại; đọc và phân tích execution log; xác thực chữ ký webhook.	"Workflow chạy lỗi ở node thứ tư — em tìm lại lần chạy đó bằng cách nào?"
E2. Power Apps + Power Automate	Nền tảng low-code	E0, cộng thêm: Power Fx; connector hoặc Dataverse; trigger và action của flow.	Delegation; quy trình phê duyệt; phân quyền theo vai trò; environment và solution; ALM và quản lý phiên bản; xử lý lỗi trong flow.	"Bảng có 50.000 dòng, màn hình của em lọc đúng không? Delegation ảnh hưởng thế nào?"
E3. Google AppSheet	Nền tảng no-code	E0, cộng thêm: mô hình dữ liệu với key và ref; biểu thức; view và action. Không yêu cầu viết code .	Slice; Bot và Automation; security filter; phân quyền theo vai trò; hoạt động ngoại tuyến và đồng bộ; báo cáo và biểu đồ.	"Người dùng A không được thấy dữ liệu của người dùng B — em chặn ở đâu, và chặn ở view có đủ không?"
E4. Supabase	Nền tảng backend (BaaS / PaaS)	Thiết kế bảng PostgreSQL; Auth; Row Level Security .	Storage; Realtime; Edge Function; policy nâng cao; migration; kiểu dữ liệu và ràng buộc.	"Nếu ai đó gọi thẳng API bằng khoá công khai, họ đọc được dữ liệu của người khác không? Vì sao?"

Ghi chú về chiều sâu của track no-code: vì không bù được bằng yêu cầu viết code, chiều sâu của nhóm AppSheet phải đến từ mô hình dữ liệu, biểu thức, security filter, phân quyền và tự động hoá của chính nền tảng. Nhóm chỉ dựng hai bảng và vài màn hình vẫn có thể qua cổng nếu hiểu và làm được phần mình đã nộp, nhưng điểm đồ án sẽ thấp do phạm vi và chất lượng kỹ thuật còn mỏng.

VI. PHẦN C — KỸ THUẬT NÂNG CAO TỰ CHỌN (TẦNG 3)

Đây là phần tạo chiều sâu và khác biệt. Các kỹ thuật ở Tầng 3 là nội dung tự chọn theo bài toán, năng lực và mục đích của nhóm; không phải "kỹ thuật cốt lõi của công nghệ" — phần cốt lõi nằm ở Tầng 1. Không có ngưỡng số lượng chung: một kỹ thuật khó được làm sâu, đúng và có số liệu có thể giá trị hơn nhiều kỹ thuật chỉ ghép vào cho đủ.

C1. Kỹ thuật Web và giao diện nâng cao

Các kỹ thuật frontend nâng cao chỉ có giá trị khi nhóm hiểu cơ chế bên dưới và dùng chúng để giải quyết một nhu cầu thật của sản phẩm.

Nhóm kỹ thuật	Thư viện / API nên dùng (2026)	Kiến thức cần hiểu	Câu hỏi tự kiểm
1. Kéo thả ★★☆	dnd-kit (chuẩn cộng đồng, ~6KB, hỗ trợ bàn phím tốt); Pragmatic drag-and-drop (Atlassian, ~3,5KB, đang chạy Jira/Trello); SortableJS, vuedraggable (Vue); HTML5 Drag and Drop API thuần. KHÔNG dùng react-beautiful-dnd — đã ngừng bảo trì, không tương thích React 19.	Pointer Events so với mouse/touch events. Cảm biến và phát hiện va chạm. Kéo được bằng bàn phím. Lưu thứ tự xuống server: đánh số lại toàn bộ hay dùng chỉ số phân số. Cập nhật lạc quan và hoàn tác khi lỗi mạng.	"Người dùng chỉ có bàn phím thì kéo thế nào?" "Kéo xong lưu xuống DB ra sao — update 1 dòng hay cả bảng?"
2. Hoạt họa bằng CSS thuần ★★☆	transition, @keyframes, transform/opacity; prefers-reduced-motion; View Transitions API (Chrome/Edge/Safari đầy đủ, Firefox một phần từ đầu 2026); scroll-driven animations (Chrome/Edge đầy đủ, Safari 17+ một phần).	Vì sao chỉ nên hoạt họa transform và opacity: hai thuộc tính này chạy trên compositor, không kích hoạt lại layout và paint. Kỹ thuật FLIP. Đo bằng tab Performance. Tôn trọng prefers-reduced-motion.	"Vì sao animate thuộc tính left thì giật, còn transform: translateX thì mượt?"
3. Hoạt họa bằng thư viện ★★☆	Motion (tên mới của Framer Motion); GSAP (toàn bộ công cụ nay miễn phí); Motion One, react-spring; Web Animations API; Lottie; AutoAnimate.	Khi nào JS mới cần thiết: cử chỉ, chuyển động vật lý kiểu lò xo, chuỗi có rẽ nhánh, hoạt họa phụ thuộc dữ liệu, hoạt họa lúc phần tử bị gỡ. Đánh đổi kích thước gói.	"Chỗ này CSS thuần làm được không? Vì sao nhóm vẫn chọn thư viện?"
4. Đồ họa 3D ★★★	Three.js; React Three Fiber + drei; TresJS (Vue); Babylon.js; thẻ <model-viewer> của Google; Spline.	Scene – camera – renderer – mesh – material – light. Định dạng glTF/GLB. Raycasting. Số lệnh vẽ và hiệu năng. Giải phóng tài nguyên để không rò rỉ bộ nhớ.	"Máy yếu bị tụt khung hình — em xử lý thế nào?" "Rời khỏi trang thì bộ nhớ WebGL đi đâu?"
5. Canvas 2D và trực quan hoá ★★☆	Canvas API; Konva.js, Fabric.js; PixiJS; D3.js; ECharts, Chart.js, Recharts, visx.	Chọn SVG hay Canvas theo số phần tử, nhu cầu tương tác, khả năng tiếp cận. devicePixelRatio (lý do hình mờ trên màn Retina). requestAnimationFrame. Tự tính hit test.	"Vẽ 10.000 điểm, em chọn SVG hay Canvas, vì sao?"
6. Cử chỉ và danh sách lớn ★★☆	Pointer Events; @use-gesture; TanStack Virtual; IntersectionObserver; kéo đổi kích thước khung; phím tắt và quản lý focus.	Vì sao render thẳng 10.000 dòng làm treo trình duyệt. Cửa sổ hiển thị và vùng đệm. Giữ vị trí cuộn khi dữ liệu đổi.	"Danh sách 50.000 dòng cuộn giật — em làm gì?"
7. Realtime và cộng tác ★★★	WebSocket hoặc SSE; cập nhật lạc quan; hiển thị người đang xem; CRDT (Yjs, Automerge); Liveblocks, PartyKit.	Xung đột khi hai người sửa cùng lúc. Xếp hàng thao tác khi mất kết nối. Phân biệt trạng thái cục bộ với trạng thái server đã xác nhận.	"Hai người sửa cùng một ô cùng lúc thì ai thắng, và vì sao?"

Nhóm kỹ thuật	Thư viện / API nên dùng (2026)	Kiến thức cần hiểu	Câu hỏi tự kiểm
8. Nội dung phong phú ★★☆	TipTap (ProseMirror), Lexical, Slate; MapLibre GL, Leaflet; WebRTC, Web Audio API.	Mô hình tài liệu của trình soạn thảo. Làm sạch HTML dán từ ngoài để chống XSS. Phân lớp bản đồ và gom cụm điểm.	"Người dùng dán nội dung từ Word vào, chuyện gì xảy ra?"
9. PWA và ngoại tuyến ★★☆	Service Worker; chiến lược cache; background sync; IndexedDB; Web Worker; WebAssembly.	Vòng đời service worker và bài toán phiên bản cũ kẹt trong cache. Cache-first so với network-first. Vì sao tính toán nặng trên luồng chính làm treo giao diện.	"Em vừa deploy bản mới, vì sao người dùng vẫn thấy bản cũ?"
10. CSS hiện đại và giao diện ★★☆	Container queries, cascade layers, custom properties, :has(), subgrid; chế độ sáng/tối; i!8n; shaden/ui, Radix, Headless UI.	Vì sao container query giải quyết được thứ media query không giải quyết được. Tổ chức token màu và khoảng cách. Tách component headless khỏi phần trang trí.	"Cùng một thẻ card đặt ở sidebar và giữa trang cần hiển thị khác nhau — em làm sao?"

LƯU Ý CHO NHÓM CHUYÊN FRONTEND

Nếu backend chỉ dùng dịch vụ có sẵn hoặc mock API, nhóm nên tạo chiều sâu ở chính frontend bằng các kỹ thuật phù hợp như tương tác phức tạp, hiệu năng, accessibility, realtime, canvas/3D, PWA... Không nhồi tính năng chỉ để tăng số lượng.

Ví dụ dạng bài thực hành cho C1

- Cho sẵn một bảng dữ liệu. Thêm kéo thả sắp xếp thứ tự, lưu lại và giữ đúng thứ tự sau khi tải lại trang.
- Thêm hoạt họa khi thêm và xoá một dòng, có xử lý prefers-reduced-motion; giải thích vì sao chọn thuộc tính đó.
- Vẽ biểu đồ cột từ một mảng dữ liệu bằng SVG hoặc Canvas, không dùng thư viện biểu đồ.
- Danh sách 20.000 dòng đang cuộn giạt — làm cho nó mượt và giải thích cách làm.
- Nhúng một file glTF cho sẵn, cho xoay và bấm vào chi tiết để hiện thông tin.
- Chuyển giao diện sang chế độ tối bằng biến CSS, không sửa từng component.

C2. Kỹ thuật hệ thống và backend nâng cao

Nhóm kỹ thuật	Công nghệ nên dùng (2026)	Kiến thức cần hiểu	Câu hỏi tự kiểm
1. Cache và tối ưu truy vấn ★★☆	Redis hoặc Valkey; cache trong tiến trình; EXPLAIN ANALYZE của PostgreSQL; index tổ hợp; phân trang bằng con trỏ.	Cache-aside so với write-through. Vòng đời và vô hiệu hoá cache. Vì sao phân trang bằng OFFSET chậm dần. Đọc được kế hoạch thực thi.	"Sửa một sản phẩm xong, cache cũ xử lý thế nào?" "Trang 1 nhanh mà trang 500 chậm — vì sao?"
2. Xử lý nền và hàng đợi ★★★	BullMQ (Node), Hangfire (.NET), Celery (Python), Laravel Queue, async (Go); RabbitMQ hoặc Kafka; tác vụ định kỳ.	Vì sao gửi email hay xuất báo cáo không nên nằm trong vòng đời request. Thử lại và backoff. Tính lũy đẳng (idempotency) — chạy hai lần không sinh hai bản ghi. Hàng đợi thư chết.	"Người dùng bấm Đặt hàng hai lần liên tiếp, hệ thống tạo mấy đơn?"
3. Realtime phía máy chủ ★★☆	WebSocket (Socket.IO, SignalR, Gorilla); SSE; Redis pub/sub để chạy nhiều tiến trình.	Chọn WebSocket hay SSE theo chiều truyền. Quản lý và dọn kết nối chết. Xác thực trên kênh WebSocket. Vì sao chạy nhiều instance thì tin không tới được người ở instance kia.	"Chạy 2 bản server thì hai người ở hai bản có nhận được tin của nhau không?"

Nhóm kỹ thuật	Công nghệ nên dùng (2026)	Kiến thức cần hiểu	Câu hỏi tự kiểm
4. Tìm kiếm nâng cao ★★☆	Full-text search của PostgreSQL; Meilisearch hoặc Typesense; Elasticsearch.	Vì sao LIKE %từ khoá% không dùng được ở quy mô thật. Chỉ mục đảo ngược. Tách từ tiếng Việt và dấu. Đồng bộ giữa CSDL chính và máy tìm kiếm.	"Tìm không dấu vẫn ra kết quả có dấu — em làm sao?"
5. Tích hợp bên thứ ba ★★☆	Cổng thanh toán (VNPay, MoMo, Stripe sandbox); gửi email; lưu trữ tệp (S3, MinIO); OAuth2.	Webhook và xác thực chữ ký. Xử lý khi dịch vụ ngoài chậm hoặc chết. Đối soát khi callback không tới. Tải tệp bằng URL có chữ ký.	"Thanh toán xong mà webhook không về thì đơn hàng ở trạng thái nào?"
6. Xử lý tệp và dữ liệu lớn ★★☆	Luồng (stream) thay vì nạp cả tệp; nhập xuất CSV/Excel theo lô; sinh PDF; nén ảnh phía máy chủ.	Vì sao đọc tệp 500MB một lần làm sập tiến trình. Xử lý theo lô và báo tiến độ. Giao dịch cho thao tác nhập hàng loạt.	"Nhập 100.000 dòng CSV, đến dòng 50.000 lỗi — dữ liệu còn lại thế nào?"
7. Kiến trúc nhiều dịch vụ ★★★	Tách một dịch vụ nhỏ; gọi qua HTTP hoặc hàng đợi; API gateway đơn giản; hợp đồng dữ liệu.	Khi nào tách và khi nào KHÔNG nên tách. Giao dịch phân tán và nhất quán cuối cùng. Lấn vệt một yêu cầu qua nhiều dịch vụ.	"Vì sao nhóm em tách dịch vụ này ra? Nếu để chung thì hỏng chỗ nào?"
8. Quan sát hệ thống ★★★	OpenTelemetry; Grafana + Prometheus; Sentry; log có trace id.	Phân biệt log, chỉ số và lần vết. Gắn trace id xuyên suốt một yêu cầu. Chọn chỉ số nào để đặt cảnh báo cho có ích.	"Một người dùng báo lỗi lúc 9h15 — em tìm lại yêu cầu đó bằng cách nào?"

Ví dụ dạng bài thực hành cho C2

- 1. Thêm cache cho một endpoint đang chậm, xử lý đúng việc vô hiệu hoá cache khi dữ liệu thay đổi.
- 2. Thêm endpoint có phân trang, lọc nhiều điều kiện và sắp xếp; giải thích index nào phục vụ truy vấn đó.
- 3. Chuyển một tác vụ nặng đang chạy đồng bộ sang chạy nền, có trạng thái theo dõi.
- 4. Thêm phân quyền: chỉ chủ sở hữu bản ghi hoặc admin mới được sửa.
- 5. Viết endpoint nhận webhook, xác thực chữ ký, xử lý trường hợp nhận trùng hai lần.

C3. Trải nghiệm và năng lực thiết bị (ứng dụng đa nền tảng)

Nhóm kỹ thuật	Nội dung và công nghệ	Câu hỏi tự kiểm
1. Hoạt họa và chuyển cảnh ★★☆	Hoạt họa ngầm định và tường minh; AnimationController; chuyển cảnh Hero; hoạt họa danh sách; Rive hoặc Lottie.	"Chuyển màn hình mà ảnh bay theo — cơ chế nào làm được?"
2. Cử chỉ ★★☆	Vuốt để xoá, kéo để làm mới, kéo thả sắp xếp, chạm để phóng to, bottom sheet kéo được.	"Vuốt ngang để xoá mà danh sách cũng cuộn ngang — em xử lý xung đột cử chỉ thế nào?"
3. Đồng bộ và ngoại tuyến ★★★	Ưu tiên dữ liệu cục bộ; hàng đợi thao tác khi mất mạng; đồng bộ lại; xử lý xung đột.	"Người dùng sửa dữ liệu khi offline, lúc có mạng lại thì sao?"
4. Phần cứng và quyền ★★☆	Camera và quét mã QR; vị trí và bản đồ; sinh trắc học; cảm biến; chọn và tải tệp.	"Người dùng từ chối cấp quyền camera thì màn hình hiện thị gì?"
5. Thông báo đẩy ★★☆	Firebase Cloud Messaging; thông báo cục bộ theo lịch; xử lý khi ứng dụng chạy nền hoặc đã tắt; deep link từ thông báo.	"Bấm vào thông báo lúc app đang tắt hẳn thì mở vào đâu?"

Nhóm kỹ thuật	Nội dung và công nghệ	Câu hỏi tự kiểm
6. Vẽ tùy biến ★★★	CustomPainter hoặc Canvas; biểu đồ tự vẽ; chữ ký tay; hiệu ứng nền.	"Vì sao phải tự vẽ thay vì ghép các widget có sẵn?"
7. Nền tảng và phát hành ★★☆	Cấu hình nhiều môi trường; deep link và app link; đa ngôn ngữ; chế độ sáng tối; cập nhật trong ứng dụng.	"Cùng một bản build, làm sao trở được sang server thử nghiệm?"

C4. Kỹ thuật AI nâng cao

Nhóm kỹ thuật	Nội dung và công nghệ	Câu hỏi tự kiểm
1. Truy hồi nâng cao ★★☆	Kết hợp tìm theo từ khoá và ngữ nghĩa; sắp xếp lại kết quả (rerank); cắt đoạn theo cấu trúc tài liệu; viết lại truy vấn.	"Câu này trả lời sai — do khâu truy hồi hay khâu sinh? Em chứng minh bằng cách nào?"
2. Gọi công cụ và tác tử ★★★	Function calling; MCP; đồ thị trạng thái; giới hạn số vòng lặp; xử lý khi mô hình gọi sai công cụ.	"Mô hình gọi công cụ xoá dữ liệu — em chặn ở đâu?"
3. Đa phương thức ★★☆	Xử lý đồng thời ảnh và văn bản: đọc hoá đơn, biểu mẫu; trích xuất thành dữ liệu có cấu trúc.	"Ảnh mờ hoặc chụp nghiêng thì kết quả thế nào?"
4. Tối ưu chi phí và độ trễ ★★☆	Cache theo câu hỏi; chọn mô hình nhỏ cho tác vụ đơn giản; gộp lô; rút gọn ngữ cảnh gửi lên.	"Chi phí mỗi truy vấn là bao nhiêu, và em giảm được bao nhiêu phần trăm?"
5. Đánh giá tự động nâng cao ★★★	Dùng mô hình chấm mô hình; bộ dữ liệu hỏi quy chạy lại mỗi lần đổi prompt; theo dõi trôi chất lượng.	"Làm sao biết prompt mới tốt hơn chứ không phải may?"
6. An toàn cho ứng dụng AI ★★★	Chống prompt injection; lọc đầu vào và đầu ra; giới hạn quyền của công cụ; chạy trong môi trường cách ly.	"Người dùng nhập một câu bảo mô hình bỏ qua mọi quy tắc — chuyện gì xảy ra?"

C5. Tự động hoá nâng cao

Nhóm kỹ thuật	Nội dung và công nghệ	Câu hỏi tự kiểm
1. Tính lũy đẳng (idempotency) và chống dữ liệu trùng ★★★	Bảo đảm một sự kiện đến hai lần không sinh hai bản ghi; khoá theo mã sự kiện; lưu dấu vết đã xử lý.	"Webhook gửi lại lần thứ hai do timeout — hệ thống tạo mấy đơn?"
2. Logic phức tạp trong nền tảng ★★☆	Với n8n: node code gọi API có phân trang, gộp dữ liệu nhiều nguồn. Với AppSheet/Power Apps: biểu thức lồng nhau, slice, delegation, Power Fx.	"Nguồn dữ liệu trả về 10 trang — em lấy đủ bằng cách nào?"
3. Ghép AI vào quy trình ★★☆	Node AI; phân loại email, trích xuất thông tin, tóm tắt; ràng buộc đầu ra để bước sau dùng được.	"Mô hình trả về sai định dạng thì các bước phía sau thế nào?"

Nhóm kỹ thuật	Nội dung và công nghệ	Câu hỏi tự kiểm
4. Tích hợp nhiều hệ thống ★★☆	Nói từ 3 hệ thống trở lên; xử lý khi một hệ thống chết; bù trừ khi một bước đã chạy còn bước sau lỗi.	"Bước gửi Telegram lỗi mà dữ liệu đã ghi vào Sheet — em xử lý sao?"
5. Quy trình dài có trạng thái ★★★	Chờ phê duyệt nhiều cấp; tạm dừng và tiếp tục; hết hạn tự động; lưu trạng thái.	"Người duyệt không bấm gì trong 3 ngày thì quy trình đang ở đâu?"
6. Vận hành có kiểm soát ★★★	Môi trường thử nghiệm tách khỏi môi trường thật; kịch bản kiểm thử cho quy trình; nhật ký kiểm toán; quản lý phiên bản.	"Em sửa quy trình đang chạy thật thế nào mà không hỏng dữ liệu?"

VII. PHẦN D — CHUẨN KỸ NGHỆ PHẦN MỀM (TẦNG 2)

Tầng 2 quy định theo mục tiêu kỹ nghệ phần mềm; cách hiện thực phụ thuộc nền tảng. Dự án code, self-host và low-code/no-code có thể dùng các cách khác nhau để đáp ứng cùng mục tiêu về phiên bản, kiểm thử, bảo mật, khả năng tái lập và tài liệu.

Vì vậy Tầng 2 được phát biểu theo mục tiêu, còn cách hiện thực hoá thay đổi theo đặc thù nền tảng.

Mục tiêu chung	Với dự án viết code	Với nền tảng low-code / no-code
Quản lý phiên bản	Git: nhánh, pull request, review chéo, xử lý xung đột. Không commit thẳng vào main.	Lịch sử phiên bản của nền tảng; xuất gói giải pháp (solution) theo mốc; ảnh chụp phân công ai xây phần nào.
Khả năng tái lập	Docker và docker compose khi phù hợp; chạy được bằng một lệnh; dữ liệu seed; tài khoản test.	Hướng dẫn nhập gói cấu hình; bản sao dữ liệu mẫu; tài khoản test có phân quyền.
Quản lý bí mật	Biến môi trường hoặc kho bí mật; không commit khoá API.	Trình quản lý credential hoặc connection của nền tảng; không dán khoá vào biểu thức.
Kiểm thử	Unit test, test API, hoặc một luồng end-to-end.	Bộ kịch bản kiểm thử có bảng kết quả; chạy thử tự động hoá với dữ liệu mẫu.
Tích hợp liên tục	GitHub Actions chạy build và test mỗi lần push.	Quy trình ALM và quản lý phiên bản của nền tảng khi có hỗ trợ; nếu không có thì ghi rõ nền tảng không hỗ trợ.
Bảo mật	OWASP Top 10 ở mức nhận biết; hash mật khẩu nếu tự quản lý tài khoản; chống SQL injection và XSS; CORS đúng.	Phân quyền theo vai trò; security filter và row-level security; giới hạn quyền của connector; kiểm soát ai thấy dữ liệu gì.
Tài liệu	README chạy lại được; tài liệu API sinh tự động.	Tài liệu mô tả mô hình dữ liệu, quy trình và phân quyền; sơ đồ luồng.
Làm việc cùng AI	Dùng AI để sinh test, giải thích lỗi, review code — và biết kiểm chứng kết quả. Khai báo trung thực trong mục AI Disclosure.	Như bên trái; ghi rõ phần cấu hình hoặc biểu thức nào do AI gợi ý.

Yêu cầu chung cho mọi nhóm: phải đọc và trích dẫn được tài liệu chính hãng của công nghệ đã chọn; AI có thể hỗ trợ học nhưng không thay thế việc kiểm chứng bằng tài liệu gốc.

VIII. PHẦN E — KỸ NGHỆ PHẦN MỀM, HẠ TẦNG VÀ CÔNG CỤ BỔ TRỢ

Những mục dưới đây không chọn làm công nghệ chính, nhưng nhóm nào đưa được một hai mục vào đồ án sẽ dễ đạt điểm cao ở phần chất lượng kỹ thuật và tối ưu.

Tên	Loại	Độ khó	Vì sao đáng thêm vào
Docker + docker compose	Công nghệ container	★★☆	Biến đồ án thành thứ chạy được bằng một lệnh — ăn thẳng điểm mục khả năng chạy lại.
GitHub Actions	Nền tảng CI/CD	★★☆	Chi phí học rất thấp, minh chứng quá trình rất mạnh: mỗi push có một lần build và test.
PostgreSQL	Hệ quản trị CSDL	★★☆	EXPLAIN, index, và extension pgvector — cho phép làm phần tối ưu có số liệu thật.
Redis / Valkey	Kho dữ liệu trong bộ nhớ / cache	★★☆	Cách nhanh nhất để có biểu đồ thời gian phản hồi trước và sau khi tối ưu.
Playwright	Framework kiểm thử	★★☆	Kiểm thử end-to-end tự động — hầu như không nhóm nào làm, làm được sẽ nổi bật.
k6	Công cụ đo tải	★★☆	Cho ra số liệu chịu tải thật thay vì nói suông về hiệu năng.
OpenTelemetry	Chuẩn quan sát (observability)	★★★	Chuẩn chung để thu thập log, chỉ số và lần vết. Khó hơn, phù hợp nhóm khá.
Grafana	Nền tảng quan sát và trực quan hoá	★★★	Hiển thị chỉ số và đặt cảnh báo. Đi kèm OpenTelemetry hoặc Prometheus.
OWASP Top 10	Khung kiến thức bảo mật	★★☆	Tự tìm và vá một lỗ hổng trong chính đồ án là nội dung chương phân tích lỗi rất tốt.

IX. CHỌN ĐÚNG CÔNG NGHỆ CHO BÀI TOÁN

Khi chọn công nghệ, nhóm phải giải thích được vì sao công nghệ đó phù hợp với bài toán và vì sao không chọn một phương án khác. Biết khi nào KHÔNG cần dùng một công nghệ cũng là một phần của năng lực kỹ thuật.

Lớp bài toán	Công nghệ phù hợp	Đừng dùng cái này — và vì sao
Hỏi đáp trên tài liệu nội bộ	Embedding + vector store + SDK mô hình (LlamaIndex hoặc tự viết)	Đừng tinh chỉnh mô hình: tốn kém, và mỗi lần tài liệu đổi lại phải làm lại từ đầu.
Dự đoán hoặc phân loại trên dữ liệu bảng	scikit-learn	Đừng dùng LLM: chậm hơn, đắt hơn và kém chính xác hơn hẳn cho loại dữ liệu này.
Trích xuất thông tin có cấu trúc từ văn bản tự do	SDK mô hình + ràng buộc đầu ra theo lược đồ (Instructor, Pydantic AI)	Đừng dựng vector store: không có bài toán tìm kiếm ở đây.
Cho một trợ lý AI truy cập được hệ thống của mình	MCP SDK	Đừng dựng agent framework nếu chỉ có vài công cụ: gọi trực tiếp đơn giản và dễ gỡ lỗi hơn.
Quy trình nhiều bước, có rẽ nhánh, cần nhớ trạng thái	LangGraph	Chuỗi tuyến tính đơn giản sẽ không đủ khi luồng cần quay lui hoặc rẽ nhánh.

Lớp bài toán	Công nghệ phù hợp	Đừng dùng cái này — và vì sao
Nhận diện đối tượng trong ảnh, số lượng lớn	YOLO (Ultralytics)	Đừng dùng LLM đa phương thức: chậm và đắt hơn nhiều lần cho tác vụ lặp lại.
Dữ liệu nhạy cảm không được gửi ra ngoài	Ollama chạy cục bộ	Đừng dùng API thương mại — đây là ràng buộc bắt buộc, không phải lựa chọn kỹ thuật.
Chuyển giọng nói thành văn bản	faster-whisper	Đừng tự huấn luyện: mô hình có sẵn đã tốt hơn nhiều so với những gì làm được trong 12 tuần.
Web nội dung, ưu tiên tốc độ tải	Astro	Đừng dùng meta-framework nặng nếu trang gần như không có tương tác.
Ứng dụng nội bộ, quy trình nghiệp vụ đơn giản, cần nhanh	AppSheet hoặc Power Apps	Đừng viết tay một hệ thống chỉ để làm lại thứ nền tảng no-code làm trong hai ngày.
Tự động hoá nối nhiều dịch vụ với nhau	n8n	Đừng viết tay toàn bộ nếu bài toán chủ yếu là nối các dịch vụ.

X. CHỦ ĐỀ MỚI NỮA CHO NHÓM MUỐN THỬ THÁCH

- Viết một MCP server cho hệ thống có thật (tra cứu lịch học, quy chế, dữ liệu thư viện) và kết nối vào một trợ lý AI.
- Xây bộ đánh giá cho một ứng dụng LLM: tập câu hỏi chuẩn, chấm tự động, báo cáo hồi quy khi đổi mô hình hoặc prompt.
- Chạy LLM cục bộ bằng Ollama và so sánh với API thương mại trên bốn trục: chi phí, độ trễ, chất lượng, tính riêng tư.
- React Compiler: bật compiler (cài babel-plugin-react-compiler và đặt reactCompiler: true — Next.js 16 hỗ trợ ổn định nhưng không bật mặc định), rồi dùng React Profiler và DevTools đo số lần render và thời gian render của một workload xác định trước và sau. Không đo bằng Lighthouse: compiler tác động vào memo hoá và số lần render, không bảo đảm cải thiện LCP.
- Triển khai edge (Cloudflare Workers, Hono) và đo độ trễ so với máy chủ truyền thống ở nhiều vị trí địa lý.
- Kotlin Multiplatform: chia sẻ tầng logic giữa Android và iOS, đo tỷ lệ mã dùng chung.
- GraphRAG với Neo4j cho dữ liệu có nhiều quan hệ thực thể.
- Bảng vẽ cộng tác nhiều người theo thời gian thực bằng CRDT (Yjs) — gộp realtime, canvas và xử lý xung đột trong một đồ án.
- Trình cấu hình sản phẩm 3D trên web bằng React Three Fiber — nhưng phải kèm phần dữ liệu và đặt hàng thật.
- Thay thư viện hoạt họa bằng CSS thuần (View Transitions + scroll-driven animations) và đo lại kích thước gói cùng chỉ số tải trước/sau.

XI. NÊN TRÁNH HOẶC CÂN NHẮC KỸ

Nhóm rủi ro	Cụ thể
Quá rộng cho 12 tuần	Kubernetes đầy đủ; hệ microservices nhiều dịch vụ; blockchain và smart contract; huấn luyện mô hình lớn từ đầu; game 3D. Nhóm sẽ dừng ở mức cấu hình được môi trường và không còn thời gian cho nội dung học thuật.

Nhóm rủi ro	Cụ thể
Quá nhẹ, không đủ chiều sâu	Chỉ HTML/CSS/Bootstrap; website giới thiệu tĩnh; ứng dụng chỉ gọi một endpoint của ChatGPT rồi in kết quả; app AppSheet hai bảng; YOLO chỉ gọi mô hình có sẵn rồi vẽ khung. Nếu vẫn muốn chọn thì phải bù bằng Tầng 3.
Sai loại khi đăng ký	Đăng ký "RAG" hoặc "MCP" làm công nghệ chính — đó là kiến trúc và giao thức, không phải framework. Đăng ký "Vite" hoặc "TanStack Query" đứng riêng — đó là công cụ build và thư viện. Đăng ký "Nuxt 3" — đã hết vòng đời hỗ trợ.
Phụ thuộc ngoài tầm kiểm soát	Đề tài cần API trả phí, GPU mạnh, thiết bị phần cứng đặc thù, hoặc dữ liệu thật của doanh nghiệp chưa xin được. Bắt buộc có phương án dự phòng (mock/stub) ngay từ tuần 2.
Trùng lặp quá nhiều	Không quá 4 nhóm cùng một công nghệ, và các nhóm cùng công nghệ phải khác miền bài toán — nếu không sẽ khó tổ chức bài thực hành tại chỗ khác nhau.

XII. BẢNG CHỌN NHANH THEO NĂNG LỰC NỀN CỦA NHÓM

Nền hiện có của nhóm	Nền chọn	Nền tránh
Mới học HTML/CSS/JS	Nuxt 4 (Vue 3); Astro + Supabase; AppSheet; Express kèm bù Tầng 3	Go, Rust, Kotlin Multiplatform, LangGraph
Đã học Java hoặc C# (OOP)	ASP.NET Core; Spring Boot; Flutter; .NET MAUI	Rust, LangGraph
Đã học Python	FastAPI + ONNX Runtime; scikit-learn; MCP SDK; LlamaIndex + pgvector; Ollama; Ultralytics YOLO; Langfuse (có thể dùng để xây RAG, MCP server, hệ thống eval — nhưng đăng ký là công nghệ, không đăng ký là pattern)	Angular, Kotlin Multiplatform
Đã khá JavaScript / React	Next.js 16; NestJS; React Native + Expo; React 19 + Vite + TanStack; Hono	Spring Boot
Nhóm giỏi, muốn thử thách	Go + Gin; Rust + Axum; Kotlin Multiplatform; Tauri 2; LangGraph; Langfuse	—

XIII. NGUỒN THAM KHẢO

- LogRocket Blog — "The 8 trends that will define web development in 2026". <https://blog.logrocket.com/8-trends-web-dev-2026/>
- O'Reilly Radar — "The AI Agents Stack (2026 Edition)". <https://www.oreilly.com/radar/the-ai-agents-stack-2026-edition/>
- Báo Thanh Niên — "Bức tranh thị trường lao động IT Việt trong năm 2026". <https://thanhnien.vn/buc-tranh-thi-truong-lao-dong-it-viet-trong-nam-2026-185251231154942235.htm>
- Model Context Protocol — tài liệu chính hãng. <https://modelcontextprotocol.io/>
- Next.js — cấu hình reactCompiler. <https://nextjs.org/docs/app/api-reference/config/next-config-js/reactCompiler>

- Nuxt — lộ trình và mốc hết vòng đời của Nuxt 3 (31/7/2026).
<https://nuxt.com/docs/3.x/community/roadmap>
- PkgPulse — "dnd-kit vs react-beautiful-dnd vs Pragmatic DnD 2026".
<https://www.pkgpulse.com/blog/dnd-kit-vs-react-beautiful-dnd-vs-pragmatic-drag-drop-2026>
- Frontend Horizon — "View Transitions API and CSS Scroll-Driven Animations: The Browser Wins of 2026". <https://www.frontendhorizon.com/blog/view-transitions-api-and-css-scroll-driven-animations-the-browser-wins-of-2026>

Lưu ý: số liệu thị trường và phiên bản thư viện thay đổi theo quý. Khi sinh viên trích dẫn trong chương Cơ hội nghề nghiệp, yêu cầu ghi rõ ngày khảo sát và nguồn cụ thể.